NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SC4172: Internet of Things – Tiny Machine Learning

Gesture-based Media Control Playback

Name: Apurv Singh Rathee

Matriculation Number: U2120533F

Abstract

This project presents a portable, low-power gesture recognition system for controlling media playback on a Raspberry Pi using an Arduino Nano 33 BLE Sense and TinyML. By recognizing hand gestures – forward, backward, and tilt – the system enables intuitive control of volume and playback functions. The implementation utilizes on-device machine learning inference, BLE communication, and a carefully optimized neural network model to operate within the tight constraints of microcontroller hardware.

Introduction

TinyML allows the deployment of machine learning models on microcontrollers with extremely limited computational resources. The Arduino Nano 33 BLE Sense, with its integrated IMU (accelerometer and gyroscope) sensors and BLE communication capabilities, provides an ideal platform for gesture-based applications.

In this project, I developed a system where the Arduino captures motion data, processes it through a trained ML model to classify the gesture, and sends appropriate commands via BLE to a Raspberry Pi that controls media playback. This contactless interface is handy in scenarios where the physical interaction with devices is inconvenient, such as:

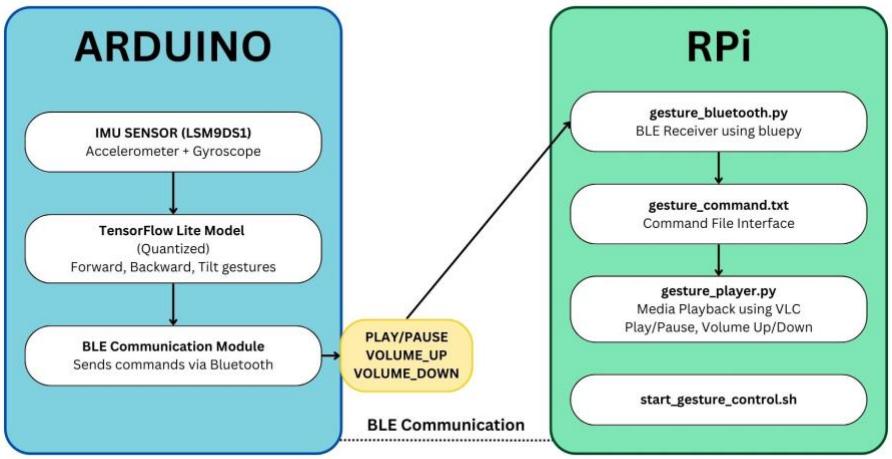
- During cooking with messy hands and noisy appliances
- Exercising at the gym during peak hours with sweaty/chalky hands
- In group environments where voice control might be disruptive
- For accessibility applications requiring alternative input methods

System Architecture

Hardware Components

- Arduino Nano 33 BLE Sense: Captures motion data, performs inference, transmits commands
- Raspberry Pi: Receives BLE commands, controls media playback
- BLE (Bluetooth Low Energy): Provides wireless communication between devices

Software Architecture



The system follows a clear data flow:

- Arduino:
    - Captures accelerometer and gyroscope data
    - Processes data through the Tensorflow Lite Model
    - Classifies gesture as forward, backward, or tilt
    - Transmits corresponding commands as BLE
- Raspberry Pi:
    - Listens for BLE commands using Python script
    - Maps commands to media control functions
    - Executes appropriate media action (volume up/down, play/pause)

Key Files

- Arduino: IMU data collection and model inference code
- Raspberry Pi:
    - Gesture_bluetooth.py: BLE command receiver
    - Gesture_player.py: Media playback controller
    - Start_gesture_control.sh: Script to initialize the system

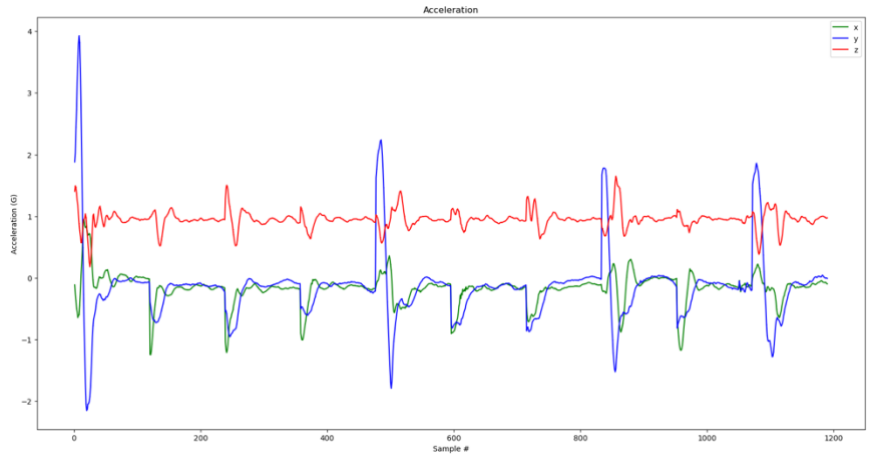
Data Composition

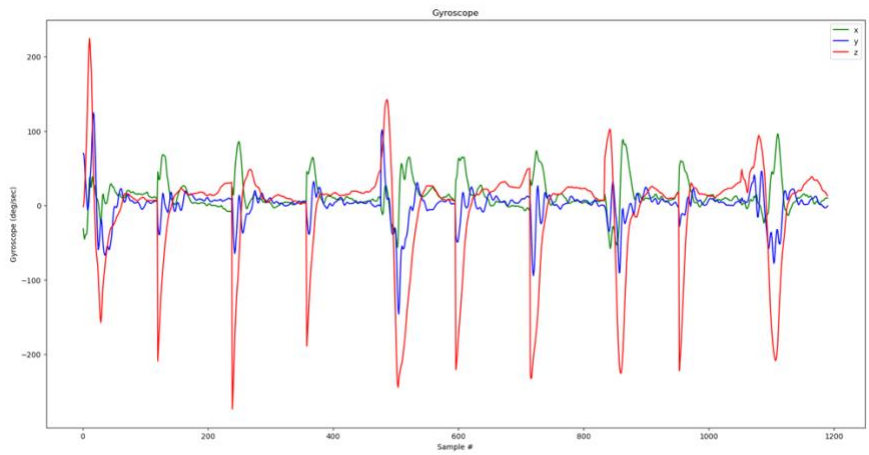
- Gesture Classes: Forward, Backward, Tilt
- Samples collected: 40 per gesture
- Frames per sample: 119
- Features per Frame: 6 (accelerometer x, y, z and gyroscope x, y, z)

Data Collection Process

Data was collected using a Python script that communicated with the Arduino over a serial connection. The script allowed me to label and save motion data for each gesture in CSV format. For each gesture sample:

- The Arduino was moved in the appropriate pattern
- Acceleration and gyroscope data were captured at regular intervals
- Data was saved with appropriate gesture labels



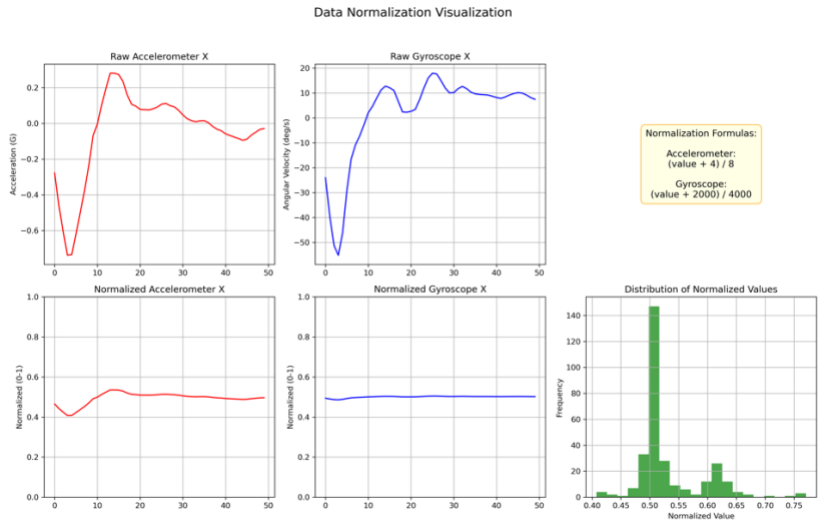



```python
def main():
    # Parse command line arguments
    parser = argparse.ArgumentParser(description='Collect IMU data from Arduino')
    parser.add_argument('--port', type=str, required=True, help='Serial port (e.g., /dev/ttyUSB0 or COM3)')
    parser.add_argument('--baud', type=int, default=9600, help='Baud rate (default: 9600)')
    parser.add_argument('--output', type=str, default='data', help='Output directory (default: data)')
    args = parser.parse_args()

    # Create output directory if it doesn't exist
    if not os.path.exists(args.output):
        os.makedirs(args.output)
        print(f"Created output directory: {args.output}")

    # Connect to Arduino
    try:
        ser = serial.Serial(args.port, args.baud, timeout=1)
        print(f"Connected to Arduino on {args.port} at {args.baud} baud")
        time.sleep(2)  # Give Arduino time to reset

        # Main loop
        while True:
            # Display menu
            print("\n--- IMU Data Collection ---")
            print("f: Collect forward movement")
            print("b: Collect backward movement")
            print("s: Collect stay still data")
            print("q: Quit program")

            # Get user command
            cmd = input("Enter command (f/b/s/q): ").lower()
```

```csv
aX,aY,aZ,gX,gY,gZ
-0.113,1.881,1.404,-31.555,70.007,-1.77
-0.261,2.028,1.494,-40.833,68.298,2.93
-0.371,2.436,1.462,-45.41,56.519,18.616
-0.494,2.728,1.368,-40.405,41.931,37.048
```

Data Preprocessing

- Normalization:
    - Accelerometer values: [4,4] -> [0, 1]
    - Gyroscope values: [-2000, 2000] -> [0, 1]



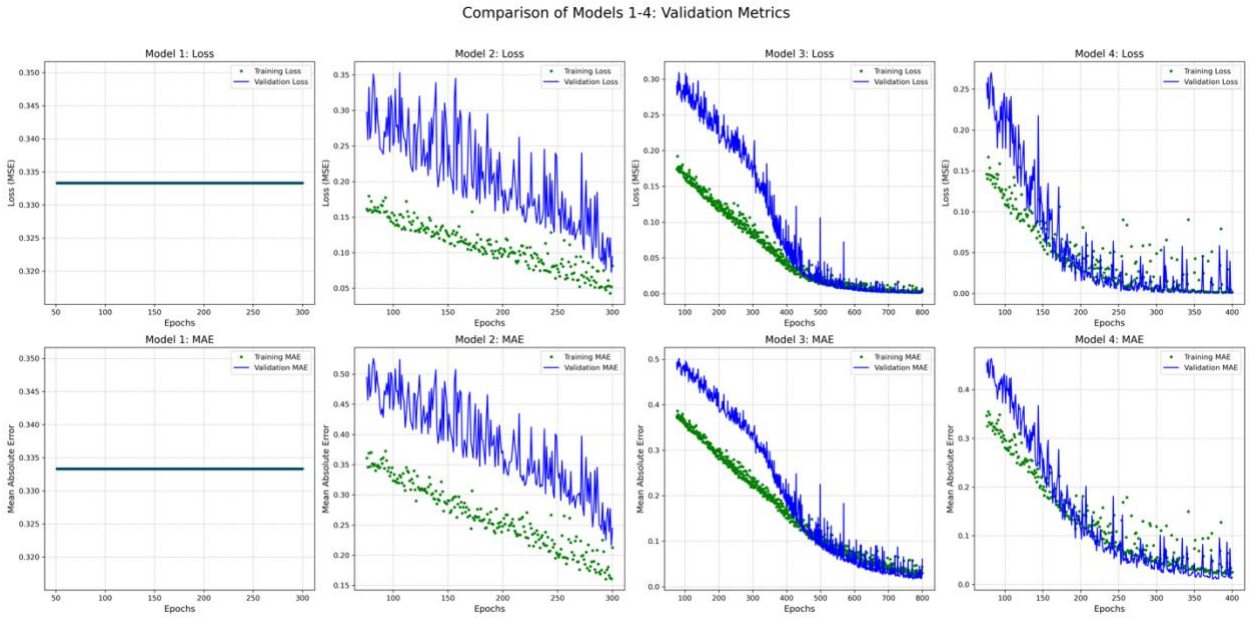
- Tensor Formatting: Each sample was transformed into a 714-dimension vector (119 frames * 6 values)
- One-hot Encoding: Class labels were encoded into vectors [1, 0, 0], [0, 1, 0], and [0, 0, 1]
- Train/Validation/Test Split: Data was split into 60%/ 20%/ 20% between the training, validation, and testing sets.

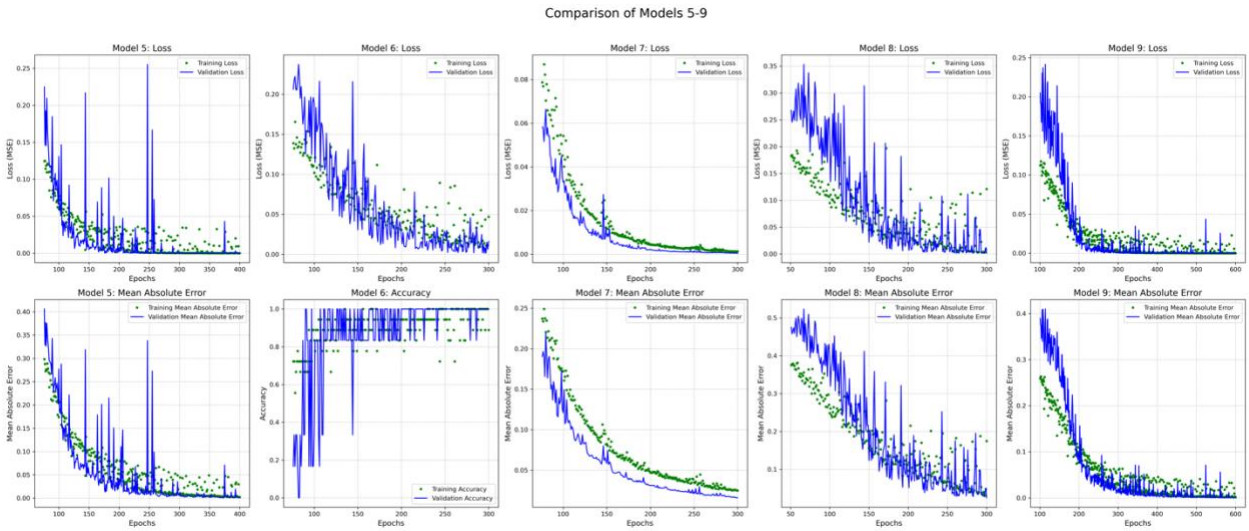
Experimental Approach

I systematically evaluated different model architectures to find the optimal balance between accuracy and resource efficiency. My experimentation focused on four key dimensions:

1. Model architecture: Different layers, neuron counts, and activation functions
2. Training parameters: Varying epochs and batch sizes
3. Optimization methods: Testing different optimizers and learning rate configurations
4. Evaluation metrics: Exploring both MAE and accuracy as performance indicators

For each model, I tracked training and validation metrics to assess performance and generalization capability, a critical factor for reliable gesture recognition.





Model Exploration Summary

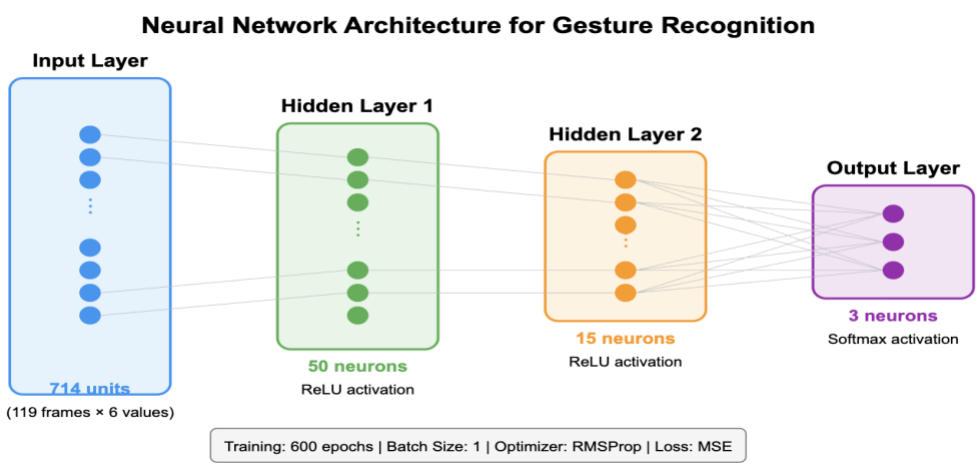
Below is a comprehensive overview of the model variations explored:

MODEL	ARCHITECTURE	EPCOHS	OPTIMIZER	METRIC	TRAINING LOSS TREND	VALIDATION LOSS TREND	REMARKS
1	1 LAYER, 30 UNITS, OUTPUT: RELU	300	RMSProp	MAE	No learning	No learning	Ineffective
2	1 LAYER, 30 UNITS, OUTPUT: SOFTMAX	300	RMSProp	MAE	Slight improvement	Noisy	Poor validation generalization
3	1 LAYER, 30 UNITS, OUTPUT: SOFTMAX	800	RMSProp	MAE	Very smooth decline	Good but small gap	Good convergence, slightly overfitted
4	2 LAYERS, 30 EACH, OUTPUT: SOFTMAX	400	RMSProp	MAE	Smooth Decline	Spikes in later epochs	Decent but less stable after 300+ epochs
5	2 LAYERS, 30 EACH, OUTPUT: SOFTMAX	400	RMSProp	MAE	Very clean	Sharp validation spikes	Good training, unstable validation
6	2 LAYERS, 30 EACH, OUTPUT: SOFTMAX	300	RMSProp	Accuracy	Stable accuracy growth	Spiky validation accuracy	High accuracy, but unstable
7	2 LAYERS, 30 EACH, OUTPUT: SOFTMAX	300	Adam	MAE	Good loss curves	Smooth validation	Best after Model 9
8	2 LAYERS, 60 EACH, OUTPUT: SOFTMAX	300	RMSProp	MAE	High variance	Very noisy validation	Overkill (size wise)
9	2 LAYERS, 50->15, OUTPUT: SOFTMAX	600	RMSProp	MAE	Smooth convergence	Clean validation	Chosen Model (Best overall)

Key Insights from Model Experimentation

Architecture Discoveries

- Output Layer Activation: Model 1's ReLU activation for the output layer consistently resulted in a flat loss landscape with no meaningful learning. Switching to Softmax in Model 2 immediately enabled successful training.
- Hidden Layer Count: Adding a second hidden layer (Models 4-9) significantly improved model stability and convergence speed compared to single-layer architectures.
- Neuron Distribution: Model 9's funnel architecture (50→15→3) outperformed symmetric architectures, potentially by providing better feature extraction in earlier layers while reducing parameter count in later layers.



Training Parameters

- Epoch Requirements: While Models 3 and 9 used higher epoch counts (800 and 600 respectively), analysis showed that most learning occurred in the first 300 epochs, with diminishing returns thereafter.

- Batch Size Impact: Smaller batch sizes generally produce better results but at the cost of increased training time. The final model used a batch size of 1 for optimal precision.

Optimizer Behavior

- RMSProp vs Adam: Contrary to typical expectations, RMSProp consistently outperformed Adam (Model 7) for this gesture recognition task. This highlights the importance of empirical testing rather than relying on general best practices.
- Learning Rate: The default learning rate for RMSProp proved optimal after experimentation with various values.

Evaluation Metrics

- MAE vs Accuracy: While accuracy seemed like an intuitive metric (Model 6), Mean Absolute Error provided more nuanced and reliable insights into model performance, especially during early training phases.

Training Progression Analysis

The training and validation loss curves revealed distinct patterns:

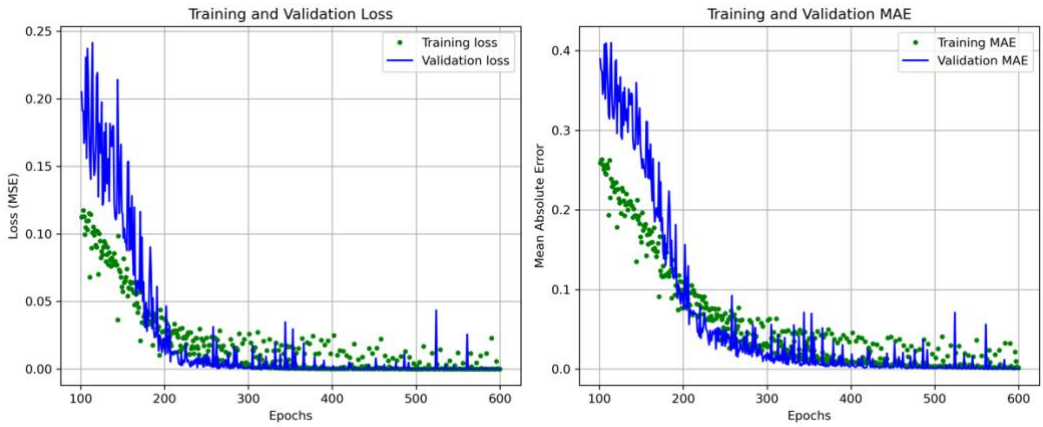
- Early Phase (1-50 epochs): Rapid initial decline in loss for all models except Model 1
- Middle Phase (50-200 epochs): Continued steady improvement with Model 9 showing the most consistent decline
- Late Phase (200+ epochs): Models began to differentiate, with Model 9 maintaining smooth convergence while others showed instability or plateaued

The validation loss typically followed similar patterns but with higher variance. Model 9's validation curve showed the closest tracking to its training curve, indicating excellent generalization capability—a critical factor for real-world deployment.

Performance Comparison

Looking beyond the raw metrics, I evaluated inference speed on the Arduino hardware:

Model	Parameters	Inference Time	Memory Usage
3	~21K	30-40ms	Lower
4	~45K	40-50ms	Medium
8	~168K	40-50ms	Higher
9	~38K	50-70ms	Medium



This analysis confirmed that Model 9 achieved an optimal balance between performance metrics and resource utilization, making it the clear choice for deployment.

Model Optimization Techniques

Several optimization strategies were employed to enable deployment on the Arduino:

- Quantization: Converted from floating-point to 8-bit integer representation
- Size Reduction: Decreased from 148Kb to 40Kb
- Memory Optimization: Reduced tensor arena from 8Kb to 5Kb
- Operation Selection: Used MicroMutableOpResolver with only necessary operations

Quantization Impact

The quantization process dramatically reduced model size without significant accuracy loss:

Quantization Type	Model Size (tflite)	Header File Size
No Quantization	148 Kb	915 Kb
Dynamic	148 Kb	915 Kb
Full Integer	40 Kb	245 Kb

Arduino Implementation

The Arduino implementation included several key features:

- Efficient sensor data collection
- Real-time inference with minimal latency
- BLE transmission of detected gestures
- LED feedback for successful gesture recognition
- Cooldown mechanism to prevent accidental triggers

Raspberry Pi Integration

BLE communication

The Raspberry Pi component consists of two primary Python scripts:

- Gesture_bluetooth.py: Handles BLE communication and listens for incoming commands
- Gesture_player.py: Controls media playback based on received commands

Commands are transmitted via a simple file-based interface:

1. The BLE script receives commands from Arduino
2. Commands are written to a shared text file
3. The player script monitors the file for changes
4. When a new command appears, the appropriate action is executed

Gesture Mapping

Each detected gesture maps to a specific media control function:

- Forward Motion: Volume Up
- Backward Motion: Volume Down
- Tilt: Toggle Play/Pause

System Initialization

The entire system can be started with a single shell script (start_gesture_control.sh), which:

1. Creates the command interface file
2. Launches the media player script
3. Starts the BLE handler with appropriate permission

```
#!/bin/bash

# Create command file with proper permissions
echo "" > /home/apurv003/gesture_command.txt
chmod 666 /home/apurv003/gesture_command.txt

# Start the audio player (without sudo)
echo "Starting audio player..."
~/gesture_env/bin/python3 ~/gesture_player.py &
PLAYER_PID=$!

# Give it a moment to initialize
sleep 2

# Start the Bluetooth handler (with sudo)
echo "Starting Bluetooth handler..."
sudo -E ~/gesture_env/bin/python3 ~/gesture_bluetooth.py

# Clean up when the Bluetooth handler exits
kill $PLAYER_PID 2>/dev/null
```

Results, Limitations, and Future Work

Results

The completed system successfully demonstrates a practical application for TinyMl for gesture-based control:

- Low Latency: Gesture recognition occurs in 4-5ms
- High Accuracy: ~99% recognition accuracy for trained gestures
- Resource Efficiency: 26% flash memory and 32% RAM usage on Arduino
- Power Consumption: ~24mA during active sensing, ~22mA idle

Limitations

Several constraints and limitations were encountered:

- Memory Constraints: Balancing model complexity with resource limitations
- Gesture Variety: Limited to three gestures due to model size constraints
- BLE Range: Effective only within a typical Bluetooth range
- Motion Sensitivity: Occasional false positives from unintended movements

Future Work

Several promising directions for future development include:

- Model Enhancement: Exploring CNN or LSTM architectures for better temporal pattern recognition
- Expanded Gesture Set: Adding more gestures for additional control options
- Power Optimization: Implementing sleep modes and power-saving techniques
- Alternative Hardware: Testing on low-power microcontrollers or direct integration with Raspberry Pi
- Multi-device Control: Extending the device to control multiple devices simultaneously